

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	1-MPI矩阵乘法	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.4.1

实验目的

- 掌握MPI矩阵乘法对矩阵分块、进行分布式计算的思想。
- 掌握MPI进程分发、广播、聚集的方法。
- 讨论大规模矩阵乘法计算和大规模系数矩阵乘法计算的优化方法。

实验过程与核心代码

分块矩阵乘法

```
void times(int *A, int *B, int *C, int n, int row_prcs) {  
    for (int i = 0; i < row_prcs; i++) {  
        for (int j = 0; j < n; j++) {  
            C[i * n + j] = 0;  
            for (int k = 0; k < n; k++) {  
                C[i * n + j] += A[i * n + k] * B[k * n + j];  
            }  
        }  
    }  
}
```

并行矩阵乘法的思路是，将其中一个矩阵按行分成多个矩阵，与另一个矩阵相乘，再将计算所得的矩阵组合起来，形成结果矩阵。

MPI进程操作

```
MPI_Init(&argc, &argv);  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

`MPI_Init(&argc, &argv)`: 初始化MPI环境, &argc传递命令行参数的个数, &argv是命令行参数的数组。

`MPI_Comm_rank(MPI_COMM_WORLD, &rank)`: 获取当前进程在MPI通信器中的进程号。
MPI_COMM_WORLD指全体进程的通信器, &rank存储当前进程的编号。

`MPI_Comm_size(MPI_COMM_WORLD, &size)`: 获取MPI通信器中进程的总数。

```

    MPI_Barrier(MPI_COMM_WORLD);
    start = MPI_Wtime();

    MPI_Scatter(A, row_prcs * n, MPI_INT, Mem_A, row_prcs * n, MPI_INT, 0,
MPI_COMM_WORLD);
    MPI_Bcast(B, n * n, MPI_INT, 0, MPI_COMM_WORLD);

    times(Mem_A, B, Mem_C, n, row_prcs);

    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Gather(Mem_C, row_prcs * n, MPI_INT, C, row_prcs * n, MPI_INT, 0,
MPI_COMM_WORLD);

    end = MPI_Wtime();

```

`MPI_Scatter(A, row_prcs * n, MPI_INT, Mem_A, row_prcs * n, MPI_INT, 0, MPI_COMM_WORLD)`：将数据从根进程中分发到所有进程中，每个进程收到 `row_prcs * n` 的数据，数据类型是 `MPI_INT`，即整型。数据存储的缓冲区是 `Mem_A`。

`MPI_Bcast(B, n * n, MPI_INT, 0, MPI_COMM_WORLD)`：广播数据，将根进程的数据发送到所有进程。广播的数据量是 `n * n`，数据类型是 `MPI_INT`，即整型。

`MPI_Barrier(MPI_COMM_WORLD)`：在所有进程之间设置一个屏障，让所有进程在此处等待，直到所有进程都到达屏障。

`MPI_Gather(Mem_C, row_per_prcs * n, MPI_INT, C, row_per_prcs * n, MPI_INT, 0, MPI_COMM_WORLD)`：将所有进程的结果收集到根进程中。每个进程传输 `row_prcs * n` 的数据，数据类型是 `MPI_INT`，即整型。数据存储的缓冲区是 `Mem_C`。

`MPI_Finalize()`：结束MPI环境使用，释放资源。

程序运行过程

1. 初始化MPI环境，获取当前进程的标识符和总进程数。
2. 计算每个进程需要处理的行数，设置缓冲区。
3. 如果当前进程是根进程，分配整个矩阵的空间，初始化矩阵数据；否则，分配当前进程需要的空间。
4. 在运行开始前同步所有进程。
5. 数据分发、数据广播。
6. 并行计算矩阵乘法。
7. 同步计算结果到主进程。
8. 结束程序，输出时间。

大规模矩阵乘法计算

内存有限情况下的大规模乘法计算可以结合分块和分布式计算。将矩阵分成小块，分别进行计算，在计算中利用分布式内存架构，将数据分布在不同节点的内存里。

大规模稀疏矩阵乘法优化

在软件层次，大规模稀疏矩阵乘法的优化主要有三个方向。

1. 使用合适的格式存储稀疏矩阵。可以使用压缩行存储或压缩列存储来存储稀疏矩阵，这分别适用于行访问频繁和列访问频繁的矩阵乘法。
2. 优化稀疏矩阵乘法算法。可以对稀疏矩阵进行矩阵的LU分解预处理矩阵，也可以采用并行化提高性能，将矩阵分成多个块，将这些块的计算拆分成独立的任务，并行执行。

实验结果

实验结果表格

进程数	矩阵规模	矩阵规模	矩阵规模	矩阵规模	矩阵规模
/	128	256	512	1024	2048
1	0.008549	0.063556	0.625629	6.971165	87.103678
2	0.007420	0.042829	0.370649	4.489649	38.808389
4	0.046680	0.053534	0.345916	3.556452	39.336443
8	0.021546	0.046265	0.429844	5.005062	42.964668
16	0.040203	0.195291	0.714279	4.467938	41.341247

随着进程数增加，运行时间整体上减少。但新建进程、回收进程也需要时间开销，因此若进程过多，运行时间反而增大。

实验感想

通过本次实验，我掌握了MPI初始化、分发进程、广播进程、收集进程的方法，学会了使用MPI点对点通信进行并行矩阵乘法。

在实验中，我遇到的最大问题是，代码运行时会报段错误，终端显示访问了未经分配的内存。经过查阅博客、仔细学习MPI相关函数，我知道了我需要在每个进程中都分配一块内存用于存储和计算。按照这样的方法解决之后，就没有再出现段错误的问题。